PROGRAMMING & CS FUNDAMENTALS

Memory Management Visual Guide

Stack vs heap, pointers, and fragmentation — where your bytes actually live.

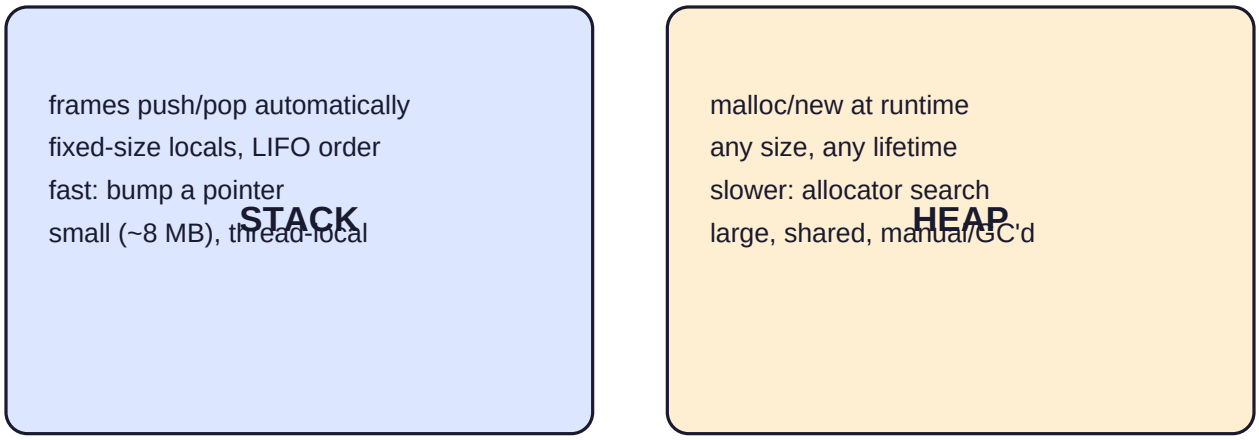
ANIMATED WALKTHROUGH

Step through pages to play the animation

STEP 1/6

Stack vs Heap

Two regions, opposite trade-offs.

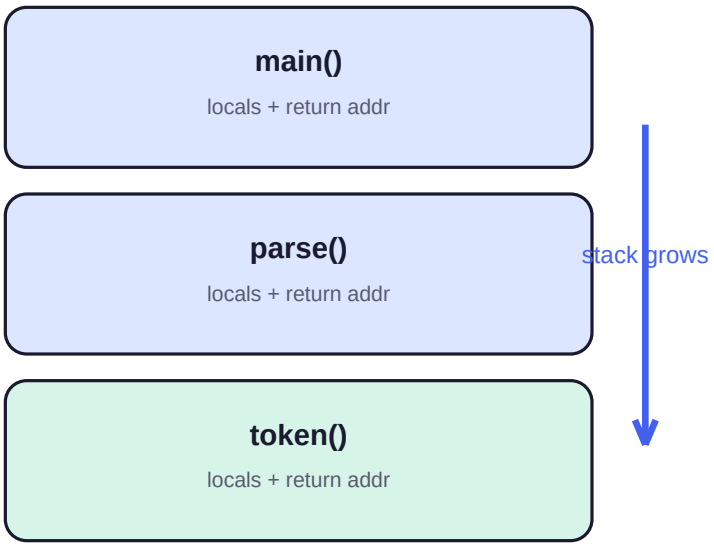


- Stack: compiler-managed, zero-cost cleanup — but fixed at compile time per frame.
- Heap: flexible lifetimes (outlive the caller) — but YOU (or GC) manage it.
- Escape: returning a stack address is instant undefined behavior — allocate on the heap instead.

STEP 2/6

Stack Frames Up Close

Every call gets a frame; returns destroy it.



token() returns -> its frame pops -> parse() resumes. Locals vanish with the frame.

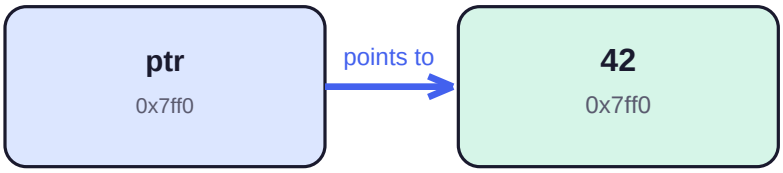
```
void f() { int x[1000000]; ... }   # 4MB on the stack: risky!
void f() { int* x = malloc(...); } # heap: safe, remember free()
```

Stack overflow = too-deep recursion or too-big locals. The stack is fast because it's dumb: no searching, just push/pop.

STEP 3/6

Pointers: Addresses as Values

A pointer is just a number — the address of something.




```
int x = 42; int* p = &x;   // p holds x's ADDRESS
*p = 43;                   // dereference: write THROUGH p
p++;                       // moves by sizeof(int), not 1 byte!
```

- Pointer arithmetic scales by type size: p+1 on int* jumps 4 bytes.
- NULL/dangling/wild: the unholy trinity — check, free once, null after free.
- References (C++/Rust) are pointers with guardrails: no arithmetic, no null (mostly).

STEP 4/6

Manual Allocation: malloc / free

With great power comes great leak-checking.

```
int* a = malloc(n * sizeof(int));  // request n ints
if (!a) handle_oom();              // malloc CAN fail
... use a[0..n-1] ...
free(a); a = NULL;                 // release exactly once
```

The four horsemen:

Leak

never freed — memory grows until OOM

Double free

freed twice — heap corruption

Use after free

dangling pointer reads garbage

Overflow

write past the end — smashes neighbors

STEP 5/6

Fragmentation

Free memory you can't use — the allocator's slow defeat.

External fragmentation:

24 bytes free total — but no 16-byte CONTIGUOUS block. Allocation fails anyway!

- Internal fragmentation: allocator rounds up (ask 24, get 32) — wasted INSIDE blocks.
- Compaction (moving GC) fixes external fragmentation — one reason GCs relocate objects.
- Slab allocators: fixed-size pools per type — no fragmentation, O(1) alloc (Linux kernel).

STEP 6/6

Memory Wisdom

Rules that prevent 3 AM pages.

- Ownership: every allocation has ONE owner responsible for freeing it — document it.
- RAII / defer / context managers: tie lifetime to scope and leaks become impossible.
- Valgrind / AddressSanitizer: run them in CI — they catch what code review misses.
- Prefer stack allocation: if it doesn't need to outlive the call, don't heap it.
- Arena allocators: free the WHOLE pool at once — perfect for per-request/per-frame data.
- Rust's borrow checker is this entire guide as compile errors — ownership made mechanical.

STEP ✓/✓

Key Takeaways

Memory Management — Remember

- Stack: fast, automatic, LIFO frames; heap: flexible lifetimes, managed allocation.
- Pointers are addresses as values — arithmetic scales by type size; null after free.
- Manual allocation's horsemen: leaks, double-free, use-after-free, overflows.
- Fragmentation (internal rounding, external scattering) wastes memory you technically have.
- Ownership + scoped cleanup (RAII/defer) + sanitizers in CI = memory sanity.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev